



QUENYX

V O P S H U B

INTERNAL

Architecture Summary — Quenyx vOPS HUB v1.0.0

Quenyx vOPS HUB — v1.0.0 · Document v1.0

Document Version	1.0
Software Version	v1.0.0
Applies To	Quenyx vOPS HUB v1.0.0 (GA)
Classification	Internal
Owner	Platform Engineering
Status	Released
Document Type	Release artifact — architecture summary

Table of Contents

- | One-paragraph summary
- | Layered view
- | Key architectural decisions
- | Module map
- | Cross-cutting guarantees

Architecture Summary — Quenyx vOPS HUB v1.0.0

One-paragraph summary

Quenyx vOPS HUB is a multi-tenant, workspace-isolated operations platform built on Laravel (API) + React/ Vite (UI). Modules are **not** wired to each other directly; they extend the platform through four **registries** (AI adapters, automation adapters, automation actions, knowledge sources) and communicate through a **Platform Event Bus**. AI is a **shared layer**: every module narrates through one `ModuleAiNarrator`, and every AI surface consumes one **Enterprise Context Engine**. The result is a single enterprise ecosystem with no duplicated AI, automation, orchestration, or context logic.

Layered view

```

UI (React/Vite)  - platformRegistry-driven nav, per-module pages, shared AI copilot drawer
                  | REST (UUID-only, workspace-scoped, RBAC)
API (Laravel)   - per-module controllers behind a uniform security envelope
                  |
Platform services
  | Registries:      AiModuleAdapterRegistry · AutomationAdapterRegistry · ActionRegistry · KnowledgeSourceRegistry
  | Event Bus:       PlatformEventBus (publish/subscribe, audited, async-ready)
  | Context Engine:  EnterpriseContextEngine (one normalized AI context)
  | AI:              ModuleAiNarrator → provider abstraction (mock-safe)
  | Read-models:     EnterpriseSearch · GlobalTimeline · KnowledgeGraph v2 · CrossModuleOrchestrator
  | Intelligence:    QynVA Operator · QynBalance Cost · Executive · Analytics · Platform Health
                  |
Data (MySQL)    - workspace-scoped tables; audit_logs; UUID addressing
  
```

Key architectural decisions

- **Registry-driven extensibility.** New module AI, execution adapters, actions, knowledge sources, and event subscribers are added by one registration line — no module branching anywhere.
- **Event-driven, decoupled.** The Platform Event Bus replaces direct module-to-module calls; publishers don't know subscribers; failures are isolated; everything is audited.
- **One context, one narrator.** The Context Engine is the single source of AI context; `ModuleAiNarrator` is the single path to providers. No direct provider calls in modules.
- **Deterministic read-models.** Search, Timeline, Graph, Analytics, Executive, and Cost are pure read-models over real rows — no duplication, no fabrication; honest `available:false` / "pricing unavailable" states.
- **QynVA orchestrates, never executes.** Cross-module coordination is an editable plan referencing existing module actions; the owning module executes after human approval.
- **Security envelope everywhere.** Workspace resolution → entitlement → RBAC
(`accessAi` / `can_use_ai` / `administerAi`) → audit, on every endpoint. UUID-only, workspace-isolated.

Module map

Module	Role
QynSight	Native monitoring + Operations Intelligence
QynAsset	Asset Intelligence
QynRun	Enterprise Automation (shared Automation Platform)
QynReact	Incident Workspace + cross-module orchestration
QynKnow / QynSupport / QynNotify	Knowledge / Service Desk / Notification Intelligence
QynShield	QCIF compliance
QynBalance	Cost Intelligence
QynVA	Enterprise AI Operator (+ Executive, Analytics, Platform Health)
QynCore	Platform core (internal communication) — not a customer module

Cross-cutting guarantees

Workspace isolation · UUID-only addressing · RBAC + entitlements · full audit trail · EN/AR i18n · mock-safe AI (real evidence regardless of provider) · no fabrication.